



POLITECNICO
MILANO 1863

Porting RTEMS on the Versal AI Edge Board

Francesco Danese



POLITECNICO
MILANO 1863

Advanced Operating Systems course project – 2025/2026

Hardware Platform: Trenz TE0950

- The hardware platform used in this project is the **Trenz TE0950**, a system-on-module built around the AMD/Xilinx **Versal AI Edge** device: a heterogeneous SoC combining programmable logic (PL), a scalar Processing System (PS), and a dedicated AI Engine array on a single die.
- The Processing System exposes two processor complexes: the **APU** (dual ARM Cortex-A72, general-purpose) and the **RPU** (dual ARM Cortex-R5F, real-time and deterministic). This project targets the RPU.



The Software Stack: RTEMS

- **RTEMS** (Real-Time Executive for Multiprocessor Systems) is an open-source RTOS designed for embedded and safety-critical systems. It provides deterministic scheduling, interrupt management, and a POSIX-compatible API, making it a solid choice for real-time applications on resource-constrained hardware.
- Applications are developed on a host machine and **cross-compiled** against the target BSP using the arm-rtems7 toolchain. The resulting ELF binary is then packaged into a bootable PDI image and deployed directly to the board over JTAG.



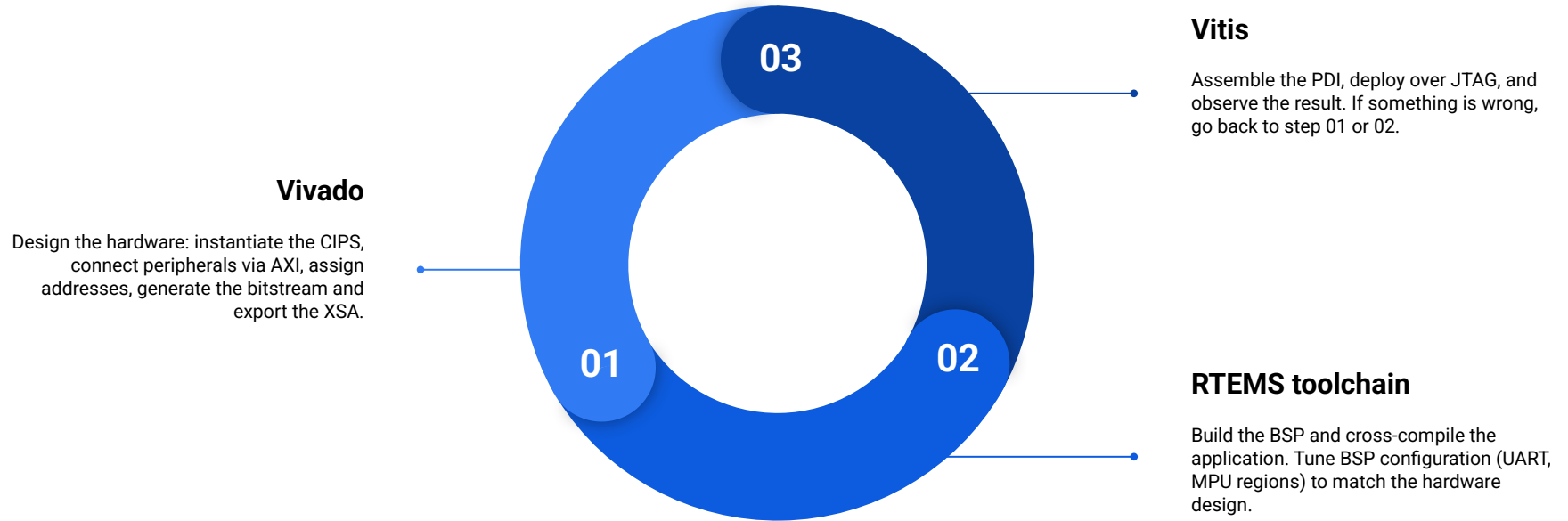
The Goals Of The Project

A BSP for the Versal RPU already existed in the RTEMS codebase, but no practical documentation or working guide was available. This project aims to fill that gap by providing a clear, reproducible workflow for the community.

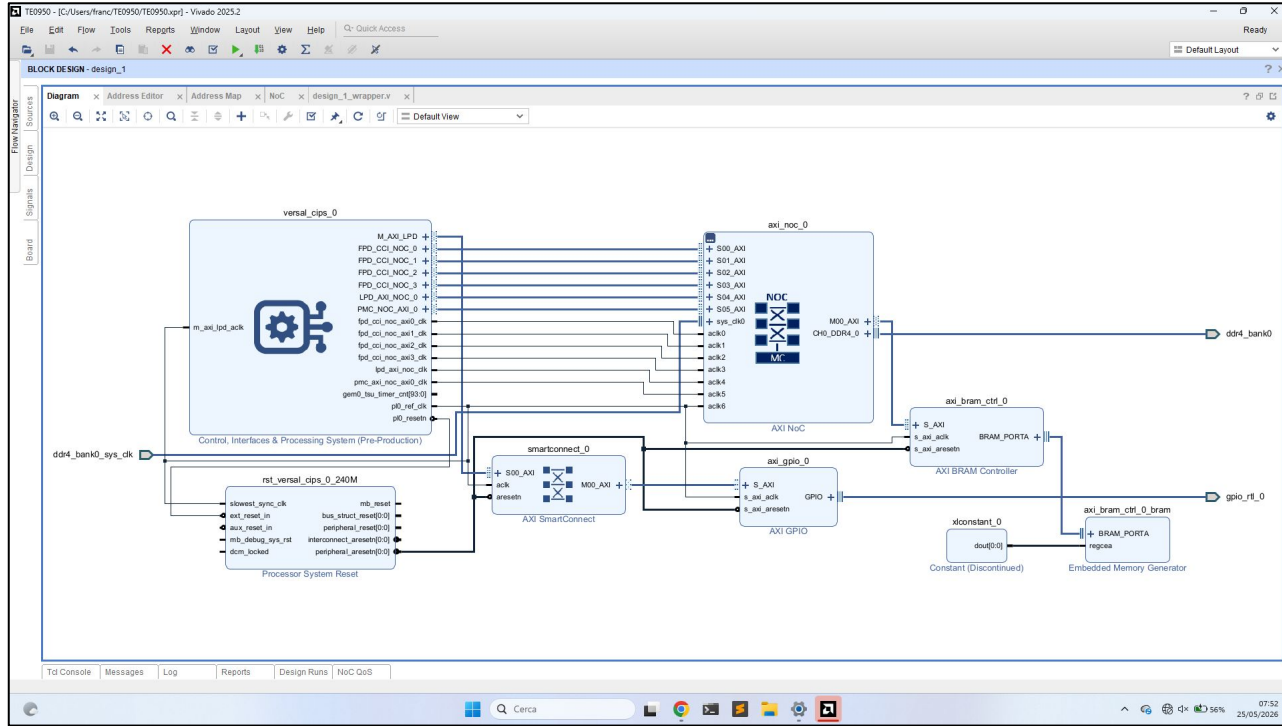
Two concrete goals were defined:

1. **Boot RTEMS on the RPU** — establish a working RTEMS configuration running on the Cortex-R5F cores in split mode, documenting every step from toolchain setup to board deployment.
2. **Drive a PL-connected LED from RTEMS** — extend the setup to control a user LED wired to the Programmable Logic via AXI GPIO, serving as a concrete, visual proof that the RTOS can interact with custom PL peripherals.

An Iterative Development Cycle



Hardware Design — Vivado Block Design



Software Design — RTEMS Configuration

BSP & System Configuration

- A **Board Support Package** provides the low-level hardware abstractions needed to boot the OS: startup code, interrupt controller, timer, and console driver
- The **Memory Protection Unit** must be explicitly configured to allow the CPU to access peripheral regions: any unmapped address triggers a fault on the Cortex-R5F
- Peripheral memory must be marked as **Device / non-cacheable** to guarantee that writes reach the hardware and are not absorbed by the cache

The Application

- RTEMS applications run as **tasks**: the Init task is the entry point, equivalent to main in a bare-metal program
- The LED is controlled via **memory-mapped I/O**: the CPU writes to a specific address that is wired to the GPIO peripheral, with no OS abstraction in between
- A **memory barrier** is required after each write to enforce ordering and ensure the store is committed to the bus before execution continues

Key Learning Outcomes

- **Low-level debugging** — using XSDB to read registers, sample the program counter to pinpoint exact failure points at the hardware level
- **Memory-mapped I/O** — understanding that a peripheral is just a set of addresses, and that the CPU must be explicitly told how to treat that memory region
- **Memory barriers** — why volatile alone is not enough on an out-of-order processor, and how a *dsb sy* instruction enforces store ordering to peripheral registers
- **Linker scripts** — how load addresses determine where code lives in memory, and how overlapping regions cause silent, hard-to-diagnose failures
- **Iterative hardware/software co-design** — every change in Vivado ripples through Vitis and the BSP configuration; the toolchain is a chain in the truest sense

Main Challenges & How We Solved Them

- **RTEMS hanging at boot** — RTEMS was spinning indefinitely inside the UART0 initialisation routine because UART0 was never enabled in hardware. Diagnosed via XSDB program counter sampling, solved by redirecting the console to UART1
- **MPU fault on GPIO access** — replacing the bare-metal application with RTEMS caused an immediate crash, because the GPIO memory region was not covered by any MPU entry. Solved by adding a Device/non-cacheable MPU region for 0x80000000

The final result is a complete, verified setup: RTEMS boots and runs on the Versal R5F cores, PS–PL communication is functional, and the entire configuration is backed by a detailed step-by-step guide for full reproducibility.